Azure APIM AI Gateway for Claude on Microsoft Foundry

Internal Runbook — Deployment, Policies, Validation & Go-Live

Purpose: Deploy, validate, troubleshoot, and operationalize the APIM-based Claude gateway on Azure, including the exact working policies, local mock backend validation path, and production cutover steps.

Audience: Internal Azure engineers, solution architects, and coding agents.

Prerequisites

Before starting, ensure the following are provisioned:

Resource	Minimum Tier	Notes
Azure API Management	Developer or Standard	Consumption tier does NOT support `llm-token-limit`
Application Insights	Any	Must be linked to APIM instance
Log Analytics Workspace	Any	Required for diagnostic settings
Azure Managed Identity	System-assigned on APIM	Required for Foundry auth when going live

> **Screenshot reference:** Azure Portal → Resource Group → verify all four resources exist before proceeding.

Architecture Overview

```
Caller (app / Claude Code / Postman)
        │
        │  Ocp-Apim-Subscription-Key: <dept-key>
        │  x-team-id: engineering
        │  Content-Type: application/json
        ▼
┌──────────────────────────────────────────────────────────┐
│           Azure API Management Instance                  │
│                                                          │
│  ┌──────────────────────────────────────────────┐        │
│  │  PRODUCT: claude-enterprise / standard /     │        │
│  │           restricted                         │        │
│  │  Policy: llm-token-limit (tier-specific)     │        │
│  └──────────────────────────────────────────────┘        │
│                      ↓ <base />                          │
│  ┌──────────────────────────────────────────────┐        │
│  │  API: Claude Foundry Gateway                 │        │
│  │  Policy: auth, headers, metrics,             │        │
│  │          synthetic headers, error format     │        │
│  └──────────────────────────────────────────────┘        │
│                      ↓ <base />                          │
│  ┌──────────────────────────────────────────────┐        │
│  │  OPERATION: POST /v1/messages                │        │
│  │  Mock Backend (return-response)              │        │
│  └──────────────────────────────────────────────┘        │
└──────────────────────────────────────────────────────────┘
        │
        ▼
  Microsoft Foundry → Claude
  (live: https://<resource>.services.ai.azure.com/anthropic)
  (mock: return-response intercepts before backend)
```

Full Policy Execution Hierarchy

When a request arrives, APIM executes policies in this order:

```
INBOUND (top → down):
  1. Global inbound
  2. Product inbound  ← llm-token-limit fires here (tier-specific limits)
  3. API inbound      ← auth, headers, estimation, metrics fire here
  4. Operation inbound ← mock return-response intercepts here (testing only)

BACKEND:
  → Skipped entirely when mock return-response is active

OUTBOUND (bottom → up):
  4. Operation outbound (none)
  3. API outbound     ← token extraction, synthetic headers, x-token-usage fi
  2. Product outbound (none)
  1. Global outbound

ON-ERROR:
  → API on-error     ← 429 and 403 custom Anthropic-format responses fire her
```

The Practical Implication of Product vs API Policy Split

- **Product policy** holds `llm-token-limit` only. This is intentional — limits differ per tier, so they must live at the scope where tiers are defined. If `llm-token-limit` were at the API level, every subscriber would get identical limits regardless of their product.
- **API policy** holds everything else: authentication, header manipulation, observability, synthetic rate limit headers, and error formatting. This logic is identical for all tiers, so it belongs at the API scope and executes once via `<base />` inheritance.
- The two scopes stack automatically. You do not call one from the other explicitly — `<base />` in the Product policy inbound triggers the API policy inbound beneath it.

Step 1 — Create the APIM Instance

1.1 Portal

1. Azure Portal → **Create a resource** → search `API Management`
2. Click **Create**
3. Fill in:

    - **Resource group:** `rg-ai-gateway`
    - **Name:** `apim-claude-gateway`
    - **Region:** East US 2 (or match your Foundry region)
    - **Organization name:** Your org name
    - **Pricing tier:** Developer (testing) or Standard (production)
4. Click **Review + Create** → **Create**
5. Wait for provisioning (~25–45 min for Developer tier)

> **Screenshot reference:** APIM → Overview page showing Gateway URL in the format `https://apim-claude-gateway.azure-api.net`

1.2 Enable Managed Identity

1. APIM → **Security** → **Managed identities**
2. Under **System assigned**, toggle **Status** to **On**
3. Click **Save**
4. Note the **Object (principal) ID** — you will need this for Foundry RBAC when going live

> **Screenshot reference:** Managed identities blade showing Status = On and the Object ID.

1.3 Link Application Insights

1. APIM → **Monitoring** → **Application Insights**
2. Click **+ Add**
3. Select your existing Application Insights instance
4. Set **Sampling** to `100%` for initial testing (reduce in production)
5. Click **Create**

> **Screenshot reference:** Application Insights blade showing the linked instance with green status.

1.4 Enable Custom Metrics in Application Insights

The `llm-emit-token-metric` policy requires custom metrics with dimensions to be enabled in App Insights:

1. Navigate to your **Application Insights** resource
2. **Usage and estimated costs** → **Custom metrics**
3. Select **With dimensions**
4. Click **Save**

> **Screenshot reference:** Custom metrics blade showing "With dimensions" selected.

1.5 Configure Diagnostic Settings

1. APIM → **Monitoring** → **Diagnostic settings**
2. Click **+ Add diagnostic setting**
3. Name: `apim-to-log-analytics`
4. Check: **Logs** → **GatewayLogs**
5. Check: **Metrics** → **AllMetrics**
6. Destination: **Send to Log Analytics workspace** → select your workspace
7. Click **Save**

> **Screenshot reference:** Diagnostic settings blade showing GatewayLogs and AllMetrics checked with Log Analytics destination.

Step 2 — Create the API (Claude Foundry Gateway)

2.1 Add the API Manually

Claude uses the Anthropic-native endpoint format (`/anthropic/v1/messages`) rather than the standard `/chat/completions` path, so the "Import from Azure AI Foundry" wizard will not map it correctly. Use manual HTTP API creation instead.

1. APIM → **APIs** → **+ Add API**
2. Under **Define a new API**, select **HTTP**
3. Fill in:
   - **Display name:** `Claude Foundry Gateway`
   - **Name:** `claude-foundry-gateway`
   - **Web service URL:** `https://<your-foundry-resource>.services.ai.azure.com/anthropic`

     > Replace `<your-foundry-resource>` with your Foundry resource name. For mock testing, this value is not called — enter a placeholder like `https://mock.placeholder.invalid/anthropic`

   - **API URL suffix:** `claude`
   - **Products:** Leave empty for now — you will assign after creating products in Step 3
4. Click **Create**

> **Screenshot reference:** HTTP API creation blade with Web service URL and API URL suffix filled in.

2.2 Add the POST Operation

1. Inside **Claude Foundry Gateway** API → click **+ Add operation**
2. Fill in:
   - **Display name:** `Messages`
   - **Name:** `messages`
   - **Method:** `POST`
   - **URL:** `/v1/messages`
3. Click **Save**

> **Screenshot reference:** Operation editor showing POST /v1/messages.

Step 3 — Create Products and Subscriptions

3.1 Product: claude-enterprise

1. APIM → **Products** → **+ Add**
2. Fill in:
   - **Display name:** `claude-enterprise`
   - **Id:** `claude-enterprise`
   - **Description:** `Enterprise tier — 50K TPM, 5M tokens/month`
   - **Published:** Checked
   - **Requires subscription:** Checked
   - **Requires approval:** Unchecked (or per your policy)
3. Under **APIs**, click **+ Add API** → select **Claude Foundry Gateway**
4. Click **Create**

> **Screenshot reference:** Product creation blade with claude-enterprise details and Claude Foundry Gateway API added.

3.2 Product: claude-standard

1. APIM → **Products** → **+ Add**
2. Fill in:
   - **Display name:** `claude-standard`
   - **Id:** `claude-standard`

- **Description:** `Standard tier — 10K TPM, 500K tokens/month`
- **Published:** Checked
- **Requires subscription:** Checked

3. Under **APIs** → **+ Add API** → select **Claude Foundry Gateway**
4. Click **Create**

3.3 Product: claude-restricted

1. APIM → **Products** → **+ Add**
2. Fill in:
   - **Display name:** `claude-restricted`
   - **Id:** `claude-restricted`
   - **Description:** `Restricted tier — 2K TPM, 100K tokens/month`
   - **Published:** Checked
   - **Requires subscription:** Checked
3. Under **APIs** → **+ Add API** → select **Claude Foundry Gateway**
4. Click **Create**

> **Screenshot reference:** Products list showing all three products with Published status.

3.4 Create Subscriptions

Create one subscription per team/application. Repeat for each consumer:

1. APIM → **Subscriptions** → **+ Add subscription**
2. Fill in:
   - **Name:** `dept-engineering` (or `dept-finance`, `app-quinn-prod`, etc.)
   - **Display name:** `Engineering Department`
   - **Scope: Product**
   - **Product:** Select `claude-standard` (or appropriate tier)
3. Click **Create**
4. After creation, click the **...** menu → **Show/hide keys** to retrieve the primary key

> **Screenshot reference:** Subscriptions list showing multiple department subscriptions each scoped to a product.

Example subscription mapping:

Subscription Name	Product	Consumer
`dept-engineering`	claude-enterprise	Engineering team
`app-quinn-prod`	claude-enterprise	QUINN production app
`dept-finance`	claude-standard	Finance department
`dept-support`	claude-standard	Support team
`key-contractors`	claude-restricted	External contractors

Step 4 — Apply Product Policies

Apply the appropriate policy to each product. The `llm-token-limit` values are the only difference between tiers.

4.1 claude-enterprise Policy

1. APIM → **Products** → `claude-enterprise` → **Policies**
2. Click the **</>** editor icon
3. Replace all content with the following and click **Save**:

```
<policies>
  <inbound>
    <base />
    <!--
      PRODUCT: claude-enterprise
      Limits: 50,000 tokens/min | 5,000,000 tokens/month
      counter-key is scoped to APIM Subscription ID so each
      subscriber gets their own independent counter.
    -->
    <llm-token-limit
      counter-key="@(context.Subscription.Id)"
      tokens-per-minute="50000"
      token-quota="5000000"
      token-quota-period="Monthly"
      estimate-prompt-tokens="true"
```

```
      remaining-tokens-header-name="x-remaining-tokens"
      remaining-tokens-variable-name="remainingTokens"
      remaining-quota-tokens-header-name="x-remaining-monthly"
      remaining-quota-tokens-variable-name="remainingQuotaTokens" />
  </inbound>
  <outbound>
    <base />
  </outbound>
  <on-error>
    <base />
  </on-error>
</policies>
```

4.2 claude-standard Policy

1. APIM → **Products** → `claude-standard` → **Policies**
2. Replace all content with:

```
<policies>
  <inbound>
    <base />
    <!--
      PRODUCT: claude-standard
      Limits: 10,000 tokens/min | 500,000 tokens/month
    -->
    <llm-token-limit
      counter-key="@(context.Subscription.Id)"
      tokens-per-minute="10000"
      token-quota="500000"
      token-quota-period="Monthly"
      estimate-prompt-tokens="true"
      remaining-tokens-header-name="x-remaining-tokens"
      remaining-tokens-variable-name="remainingTokens"
      remaining-quota-tokens-header-name="x-remaining-monthly"
      remaining-quota-tokens-variable-name="remainingQuotaTokens" />
  </inbound>
  <outbound>
```

```
    <base />
  </outbound>
  <on-error>
    <base />
  </on-error>
</policies>
```

4.3 claude-restricted Policy

1. APIM → **Products** → `claude-restricted` → **Policies**
2. Replace all content with:

```
<policies>
  <inbound>
    <base />
    <!--
      PRODUCT: claude-restricted
      Limits: 2,000 tokens/min | 100,000 tokens/month
    -->
    <llm-token-limit
      counter-key="@(context.Subscription.Id)"
      tokens-per-minute="2000"
      token-quota="100000"
      token-quota-period="Monthly"
      estimate-prompt-tokens="true"
      remaining-tokens-header-name="x-remaining-tokens"
      remaining-tokens-variable-name="remainingTokens"
      remaining-quota-tokens-header-name="x-remaining-monthly"
      remaining-quota-tokens-variable-name="remainingQuotaTokens" />
  </inbound>
  <outbound>
    <base />
  </outbound>
  <on-error>
    <base />
  </on-error>
</policies>
```

> **Screenshot reference:** Policy editor for claude-standard showing the llm-token-limit block with tokens-per-minute="10000".

Step 5 — Apply the API Policy

This is the complete policy applied at the **API level** (Claude Foundry Gateway), not at a product or operation level. It handles authentication, header management, observability, synthetic Anthropic-compatible headers, and error formatting. Token limits are intentionally absent here — they live in the Product policies above.

1. APIM → **APIs** → **Claude Foundry Gateway**
2. Click **All operations** (not a specific operation — this scopes the policy to the whole API)
3. In the **Inbound processing** section, click the **</>** editor icon
4. Replace all content with the following complete policy and click **Save**:

```
<policies>
  <inbound>
    <base />

    <!--
      ============================================================
      SECTION A: AUTHENTICATION
      Use Managed Identity to authenticate to Foundry.
      Requires: APIM system-assigned MI granted
      "Cognitive Services User" role on the Foundry resource.
      ============================================================
    -->
    <authentication-managed-identity resource="https://cognitiveservices.azur

    <!--
      ============================================================
      SECTION B: REQUIRED ANTHROPIC HEADERS
      anthropic-version is mandatory on every request to the
      Anthropic Messages API. Without it, Foundry returns 400.
      ============================================================
    -->
    <set-header name="anthropic-version" exists-action="override">
      <value>2023-06-01</value>
```

```
    </set-header>

    <set-header name="Content-Type" exists-action="override">
      <value>application/json</value>
    </set-header>

    <!--
      ============================================================
      SECTION C: STRIP NATIVE ANTHROPIC RATE LIMIT HEADERS
      Foundry does not pass through Anthropic's native rate
      limit headers. Strip any that callers may have sent
      inbound to prevent confusion. APIM reconstructs them
      outbound from its own tracking data (Section H below).
      ============================================================
    -->
    <set-header name="anthropic-ratelimit-tokens-limit" exists-action="delete
    <set-header name="anthropic-ratelimit-tokens-remaining" exists-action="de
    <set-header name="anthropic-ratelimit-tokens-reset" exists-action="delete
    <set-header name="anthropic-ratelimit-input-tokens-limit" exists-action="
    <set-header name="anthropic-ratelimit-input-tokens-remaining" exists-acti
    <set-header name="anthropic-ratelimit-input-tokens-reset" exists-action="
    <set-header name="anthropic-ratelimit-output-tokens-limit" exists-action=
    <set-header name="anthropic-ratelimit-output-tokens-remaining" exists-act
    <set-header name="anthropic-ratelimit-output-tokens-reset" exists-action=
    <set-header name="anthropic-ratelimit-requests-limit" exists-action="dele
    <set-header name="anthropic-ratelimit-requests-remaining" exists-action="
    <set-header name="anthropic-ratelimit-requests-reset" exists-action="dele

    <!--
      ============================================================
      SECTION D: CAPTURE TEAM/DEPARTMENT IDENTITY
      x-team-id is a caller-supplied header used for metric
      dimensions and chargeback reporting in App Insights.
      NOTE: This is for OBSERVABILITY only, not enforcement.
      Enforcement uses context.Subscription.Id (Product policy).
      ============================================================
    -->
    <set-variable name="dept"
```

```
    value="@(context.Request.Headers.GetValueOrDefault(&quot;x-team-id&quot

  <!--
    ============================================================
    SECTION E: PRE-FLIGHT PROMPT TOKEN ESTIMATION
    Estimates input token count from request body before
    the call completes. Used to populate the
    anthropic-ratelimit-tokens-remaining synthetic header
    with a reasonable value on the outbound leg.
    Approximation: ~3.8 characters per token.
    ============================================================
  -->
  <set-variable name="estInputTokens"
    value="@{
      try {
        var body = context.Request.Body.As&lt;JObject&gt;(true);
        var msgs = body[&quot;messages&quot;]?.ToString() ?? &quot;&quot;;
        var sys  = body[&quot;system&quot;]?.ToString() ?? &quot;&quot;;
        return (int)((msgs.Length + sys.Length) / 3.8);
      } catch { return 0; }
    }" />

  <!--
    ============================================================
    SECTION F: TOKEN METRICS EMISSION
    Sends Total Tokens, Prompt Tokens, Completion Tokens
    to Application Insights as custom metrics under the
    ClaudeUsage namespace. Query in Log Analytics:
      customMetrics | where name startswith "ClaudeUsage"
    NOTE: Max 5 custom dimensions per policy (Azure Monitor limit).
    ============================================================
  -->
  <llm-emit-token-metric namespace="ClaudeUsage">
    <dimension name="API ID" />
    <dimension name="Subscription ID" value="@((string)context.Subscription
    <dimension name="Client IP" value="@(context.Request.IpAddress)" />
    <dimension name="Model" value="claude-sonnet-4-6" />
    <dimension name="Team" value="@((string)context.Variables[&quot;dept&qu
```

```
    </llm-emit-token-metric>

  </inbound>

  <outbound>
    <base />

    <!--
      ============================================================
      SECTION G: EXTRACT ACTUAL TOKEN USAGE FROM RESPONSE
      Claude returns usage in the response body as:
        { "usage": { "input_tokens": N, "output_tokens": N } }
      We extract these to populate headers and for logging.
      Body is read with preserveContent=true so the response
      body is still forwarded to the caller intact.
      ============================================================
    -->
    <set-variable name="inputTokens"
      value="@{
        try {
          var body = context.Response.Body.As&lt;JObject&gt;(true);
          return body?[&quot;usage&quot;]?[&quot;input_tokens&quot;]?.ToStrin
        } catch { return &quot;0&quot;; }
      }" />

    <set-variable name="outputTokens"
      value="@{
        try {
          var body = context.Response.Body.As&lt;JObject&gt;(true);
          return body?[&quot;usage&quot;]?[&quot;output_tokens&quot;]?.ToStri
        } catch { return &quot;0&quot;; }
      }" />

    <!--
      ============================================================
      SECTION H: SYNTHESIZE ANTHROPIC-COMPATIBLE RATE LIMIT HEADERS
      Foundry strips all anthropic-ratelimit-* headers natively.
      APIM reconstructs them from its own tracking data so that
```

```
      callers and SDKs that key off these headers continue to
      work identically to calling Anthropic directly.

      Accuracy note:
        - tokens-limit: exact (hardcoded from product tier)
        - tokens-remaining: approximate (APIM counter, not Anthropic-precise)
        - tokens-reset: approximate (UtcNow + 60s window estimate)

      The product tier limit values are resolved dynamically
      using context.Product.Name so the headers accurately
      reflect the caller's subscription entitlement.
      ============================================================
    -->
    <set-variable name="productTpmLimit"
      value="@(context.Product?.Name == &quot;claude-enterprise&quot; ? &quot
               context.Product?.Name == &quot;claude-standard&quot;   ? &quot

    <set-header name="anthropic-ratelimit-requests-limit" exists-action="over
      <value>@((string)context.Variables[&quot;productTpmLimit&quot;])</value
    </set-header>

    <set-header name="anthropic-ratelimit-requests-remaining" exists-action="
      <value>@(context.Variables.ContainsKey(&quot;remainingTokens&quot;) ? &
    </set-header>

    <set-header name="anthropic-ratelimit-requests-reset" exists-action="over
      <value>@(DateTimeOffset.UtcNow.AddSeconds(60).ToString(&quot;yyyy-MM-dd
    </set-header>

    <set-header name="anthropic-ratelimit-tokens-limit" exists-action="overri
      <value>@((string)context.Variables[&quot;productTpmLimit&quot;])</value
    </set-header>

    <set-header name="anthropic-ratelimit-tokens-remaining" exists-action="ov
      <value>@(context.Variables.ContainsKey(&quot;remainingTokens&quot;) ? &
    </set-header>

    <set-header name="anthropic-ratelimit-tokens-reset" exists-action="overri
```

```
      <value>@(DateTimeOffset.UtcNow.AddSeconds(60).ToString(&quot;yyyy-MM-dd
    </set-header>

    <!--
      ============================================================
      SECTION I: CUSTOM TRANSPARENCY HEADERS
      These are non-standard headers that provide additional
      visibility to callers beyond the Anthropic-compatible set.
      ============================================================
    -->

    <!-- Actual token consumption from this specific request -->
    <set-header name="x-token-usage" exists-action="override">
      <value>@($&quot;{context.Variables[&quot;inputTokens&quot;]}in/{context
    </set-header>

    <!-- Remaining TPM allowance in current window -->
    <set-header name="x-remaining-tokens" exists-action="skip">
      <value>@(context.Variables.ContainsKey(&quot;remainingTokens&quot;) ? &
    </set-header>

    <!-- Remaining monthly quota -->
    <set-header name="x-remaining-monthly" exists-action="skip">
      <value>@(context.Variables.ContainsKey(&quot;remainingQuotaTokens&quot;;
    </set-header>

    <!-- Which product tier this subscription belongs to -->
    <set-header name="x-product-tier" exists-action="override">
      <value>@(context.Product?.Name ?? &quot;unknown&quot;)</value>
    </set-header>

    <!-- Gateway traceability tag for support/debugging -->
    <set-header name="x-gateway" exists-action="override">
      <value>apim-claude-foundry</value>
    </set-header>

    <!-- Correlation ID for cross-service request tracing -->
    <set-header name="x-gateway-request-id" exists-action="override">
```

```
      <value>@(context.RequestId.ToString())</value>
    </set-header>

  </outbound>

  <on-error>

    <!--
      ============================================================
      SECTION J: ERROR RESPONSES IN ANTHROPIC FORMAT
      llm-token-limit returns:
        429 Too Many Requests → TPM rate limit exceeded
        403 Forbidden         → Monthly quota exceeded
      Both are returned with Anthropic-compatible JSON error
      bodies so SDK error handlers work without modification.
      ============================================================
    -->
    <choose>

      <!-- TPM rate limit exceeded -->
      <when condition="@(context.Response.StatusCode == 429)">
        <return-response>
          <set-status code="429" reason="Too Many Requests" />
          <set-header name="Content-Type" exists-action="override">
            <value>application/json</value>
          </set-header>
          <set-header name="Retry-After" exists-action="override">
            <value>60</value>
          </set-header>
          <set-body>@{
            return new JObject(
              new JProperty("error", new JObject(
                new JProperty("type", "rate_limit_error"),
                new JProperty("code", "RateLimitExceeded"),
                new JProperty("message",
                  $"Token rate limit exceeded for subscription '{context.Subs
                  $"Retry after 60 seconds."),
                new JProperty("department", context.Variables["dept"]),
```

```
                new JProperty("product", context.Product?.Name),
                new JProperty("gateway", "apim-claude-foundry"),
                new JProperty("retry_after_seconds", 60)
              ))
            ).ToString();
          }</set-body>
        </return-response>
      </when>

      <!-- Monthly quota exceeded -->
      <when condition="@(context.Response.StatusCode == 403)">
        <return-response>
          <set-status code="403" reason="Forbidden" />
          <set-header name="Content-Type" exists-action="override">
            <value>application/json</value>
          </set-header>
          <set-body>@{
            return new JObject(
              new JProperty("error", new JObject(
                new JProperty("type", "quota_exceeded_error"),
                new JProperty("code", "MonthlyQuotaExceeded"),
                new JProperty("message",
                  $"Monthly token quota exceeded for team '{context.Variables
                  $"Quota resets at the start of next month."),
                new JProperty("department", context.Variables["dept"]),
                new JProperty("product", context.Product?.Name),
                new JProperty("gateway", "apim-claude-foundry")
              ))
            ).ToString();
          }</set-body>
        </return-response>
      </when>

    </choose>
    <base />
```

```
  </on-error>
</policies>
```

> **Screenshot reference:** API policy editor showing the full policy applied to "All operations" with the inbound, outbound, and on-error sections visible.

Step 6 — Operation Mock for Fast Smoke Testing Only

Use the operation-level `return-response` mock only for quick, low-friction smoke tests. It is useful for confirming that inbound policy logic runs and that the gateway can emit the basic quota headers returned directly by `llm-token-limit`.

It is **not** the recommended way to validate the full outbound header set. In practice, `return-response` short-circuits the normal backend/response flow and does not reliably reproduce the same behavior as a real backend response for synthetic outbound headers. The fully verified path is in Step 7.

Important: If you use this block, add it at the **Operation level** (POST /v1/messages), not the API level.

1. APIM → **APIs** → **Claude Foundry Gateway** → click **POST Messages** (the specific operation)
2. In **Inbound processing**, click the **</>** editor icon
3. Replace all content with the following and click **Save**:

```
<policies>
  <inbound>
    <base />

    <!--
      ═══════════════════════════════════════════════════════
      MOCK BACKEND — OPTION B (return-response)

      Intercepts the request before it reaches the Foundry
      backend and returns a hardcoded Anthropic Messages API
      shaped response for smoke testing.

      The mock:
```

```
      - Uses a unique ID per request (context.RequestId)
      - Sets input_tokens from the pre-flight estimate
        calculated in the API inbound policy (estInputTokens)
        so the token counter reflects real prompt size
      - Sets output_tokens to a fixed 25 (simulates completion)
      - Mirrors the exact Anthropic Messages API response shape
        so usage.input_tokens and usage.output_tokens are
        where the outbound policy expects them

    TO RUN FULL OUTBOUND VALIDATION: Replace this with <base />
    and use the local mock backend workflow in Step 7.

    TO GO LIVE: Delete this entire operation policy and leave
    only <base /> in the operation scope.
    ═══════════════════════════════════════════════════════════
  -->
  <return-response>
    <set-status code="200" reason="OK" />
    <set-header name="Content-Type" exists-action="override">
      <value>application/json</value>
    </set-header>
    <set-body>@{
      var requestId = context.RequestId.ToString().Replace("-", "").Substri
      var inputEst  = (int)context.Variables.GetValueOrDefault("estInputTok

      return new JObject(
        new JProperty("id",    "msg_mock_" + requestId),
        new JProperty("type",  "message"),
        new JProperty("role",  "assistant"),
        new JProperty("model", "claude-sonnet-4-6"),
        new JProperty("content", new JArray(
          new JObject(
            new JProperty("type", "text"),
            new JProperty("text",
              "This is a mocked Claude response for APIM policy testing. "
              "Request correlation ID: " + context.RequestId.ToString() + "
              "Product tier: " + (context.Product?.Name ?? "unknown") + ".
              "Team: " + context.Variables.GetValueOrDefault("dept", "defau
```

```
            )
          )),
          new JProperty("stop_reason",   "end_turn"),
          new JProperty("stop_sequence", null),
          new JProperty("usage", new JObject(
            new JProperty("input_tokens",  inputEst > 0 ? inputEst : 42),
            new JProperty("output_tokens", 25)
          ))
        ).ToString();
      }</set-body>
    </return-response>

  </inbound>
  <outbound>
    <base />
  </outbound>
  <on-error>
    <base />
  </on-error>
</policies>
```

> **Screenshot reference:** Operation-level policy editor (POST Messages) showing the return-response block. Use this for smoke testing only.

Step 7 — Full-Fidelity Validation with a Local Mock Backend

This is the verified path for testing the complete outbound policy behavior without a live Foundry Claude deployment. Unlike the operation-level `return-response` mock, this path forces APIM to call a real backend URL and then execute the normal outbound pipeline against the backend response.

7.1 Reset the Operation Policy to Pass Through

Before using a real mock backend, remove the operation-level `return-response` block so APIM can forward traffic normally.

APIM → **APIs** → **Claude Foundry Gateway** → **POST Messages** → **Policies** and replace the operation policy with:

```
<policies>
  <inbound>
  <base />
  </inbound>
  <outbound>
  <base />
  </outbound>
  <on-error>
  <base />
  </on-error>
</policies>
```

7.2 Run a Local Claude-Shaped Mock Server

Create a local file named `local-claude-mock-server.py` with the following contents:

```
from http.server import BaseHTTPRequestHandler, HTTPServer
import json


class Handler(BaseHTTPRequestHandler):
  def do_GET(self):
    if self.path in ('/', '/healthz'):
      payload = {'status': 'ok'}
      body = json.dumps(payload).encode('utf-8')
      self.send_response(200)
      self.send_header('Content-Type', 'application/json')
      self.send_header('Content-Length', str(len(body)))
      self.end_headers()
      self.wfile.write(body)
      return

    self.send_error(404)
```

```
  def do_POST(self):
    if self.path not in ('/v1/messages', '/anthropic/v1/messages'):
      self.send_error(404)
      return

    length = int(self.headers.get('Content-Length', '0'))
    request_bytes = self.rfile.read(length)

    try:
      request_json = json.loads(request_bytes.decode('utf-8')) if request_byt
    except json.JSONDecodeError:
      request_json = {}

    messages = request_json.get('messages', [])
    prompt_text = json.dumps(messages)
    input_tokens = max(1, int(len(prompt_text) / 4))

    response_json = {
      'id': 'msg_mock_local',
      'type': 'message',
      'role': 'assistant',
      'model': request_json.get('model', 'claude-sonnet-4-6'),
      'content': [
        {
          'type': 'text',
          'text': (
            'This is a local mock backend response for APIM outbound '
            'policy testing. Request id: local-mock. '
            f'Observed prompt length: {len(prompt_text)} characters.'
          ),
        }
      ],
      'stop_reason': 'end_turn',
      'stop_sequence': None,
      'usage': {
        'input_tokens': input_tokens,
        'output_tokens': 25,
      },
```

```
    }

    body = json.dumps(response_json).encode('utf-8')
    self.send_response(200)
    self.send_header('Content-Type', 'application/json')
    self.send_header('Content-Length', str(len(body)))
    self.end_headers()
    self.wfile.write(body)


if __name__ == '__main__':
  server = HTTPServer(('127.0.0.1', 5051), Handler)
  print('Claude mock server listening on http://127.0.0.1:5051')
  server.serve_forever()
```

Run it locally:

```
python .\local-claude-mock-server.py
```

7.3 Expose the Local Mock Publicly

APIM needs a publicly reachable HTTPS backend URL. The simplest temporary options are a public tunnel such as Cloudflare Quick Tunnels or LocalTunnel.

Cloudflare Quick Tunnel example:

```
cloudflared tunnel --url http://127.0.0.1:5051
```

This returns a temporary URL similar to:

```
https://example-name.trycloudflare.com
```

LocalTunnel example:

```
npx localtunnel --port 5051
```

This returns a temporary URL similar to:

```
https://example-name.loca.lt
```

7.4 Point the APIM Backend at the Tunnel

APIM → **APIs** → **Claude Foundry Gateway** → **Settings** and set the **Web service URL** to:

```
https://<your-tunnel-host>/anthropic
```

Example:

```
https://example-name.trycloudflare.com/anthropic
```

The operation remains `POST /v1/messages`, so APIM forwards to:

```
https://<your-tunnel-host>/anthropic/v1/messages
```

7.5 Why This Path Works

- APIM performs a real backend HTTPS request.
- The local mock returns a real JSON body with `usage.input_tokens` and `usage.output_tokens`.
- The API-level outbound policy reads that body and synthesizes the Anthropic-compatible headers.
- This is the closest possible validation path to a live Foundry backend without needing Claude deployment access.

Step 8 — Verify the Setup

8.1 Test via APIM Test Console

1. APIM → **APIs** → **Claude Foundry Gateway** → **POST Messages** → **Test** tab
2. Under **Headers**, add:
    - `x-team-id` : `engineering`
3. Under **Request body**, paste:

```
{
  "model": "claude-sonnet-4-6",
  "max_tokens": 100,
  "messages": [
    {
      "role": "user",
      "content": "Hello, this is a policy test. Please confirm you are workin
    }
  ]
}
```

4. Under **Subscription**, select one of your department subscriptions (e.g. `dept-engineering`)
5. Click **Send**

8.2 Expected Response Headers

Verify the following headers are present in the response:

Header	Expected Value	Notes
`anthropic-ratelimit-tokens-limit`	`10000`	Matches claude-standard product
`anthropic-ratelimit-tokens-remaining`	`10000` or a decreasing number	APIM counter
`anthropic-ratelimit-tokens-reset`	ISO timestamp ~60s ahead	Approximate
`anthropic-ratelimit-requests-limit`	`10000`	Same as tokens limit
`anthropic-ratelimit-requests-remaining`	`10000` or a decreasing number	APIM counter
`x-token-usage`	`Nin/25out`	Derived from backend response body

Header	Expected Value	Notes
`x-remaining-tokens`	Same as ratelimit-remaining	Duplicate for transparency
`x-remaining-monthly`	`500000` for claude-standard	Monthly quota counter
`x-product-tier`	`claude-standard`	Product context
`x-gateway`	`apim-claude-foundry`	Gateway tag
`x-gateway-request-id`	GUID	Correlation ID

Verified raw response example from the working deployment:

```
HTTP/1.1 200 OK
anthropic-ratelimit-requests-limit: 10000
anthropic-ratelimit-requests-remaining: 10000
anthropic-ratelimit-requests-reset: 2026-04-10T04:11:34Z
anthropic-ratelimit-tokens-limit: 10000
anthropic-ratelimit-tokens-remaining: 10000
anthropic-ratelimit-tokens-reset: 2026-04-10T04:11:34Z
x-token-usage: 8in/25out
x-remaining-tokens: 10000
x-remaining-monthly: 500000
x-product-tier: claude-standard
x-gateway: apim-claude-foundry
x-gateway-request-id: 0d746fba-a38b-45ca-9e6f-dbd692392448
```

8.3 Test 429 Rate Limit Enforcement

To trigger a 429 from the `claude-restricted` product (2K TPM limit is easiest to hit in testing):

1. Create a test subscription scoped to `claude-restricted`
2. Send rapid repeated requests using the test console or Postman
3. After the TPM counter is exhausted, you should receive:

```
HTTP 429 Too Many Requests
Retry-After: 60

{
  "error": {
    "type": "rate_limit_error",
    "code": "RateLimitExceeded",
    "message": "Token rate limit exceeded for subscription 'key-contractors'.
    "department": "engineering",
    "product": "claude-restricted",
    "gateway": "apim-claude-foundry",
    "retry_after_seconds": 60
  }
}
```

> **Screenshot reference:** Test console showing HTTP 429 response with the Anthropic-format error body.

8.4 Verify Token Metrics in Application Insights

1. Navigate to your **Application Insights** resource
2. **Logs** → run the following KQL query:

```
customMetrics
| where name startswith "ClaudeUsage"
| extend team = tostring(customDimensions["Team"])
| extend product = tostring(customDimensions["Subscription ID"])
| summarize
    totalTokens = sum(valueSum),
    requestCount = count()
  by team, bin(timestamp, 5m)
| order by timestamp desc
```

You should see rows appearing within 1–2 minutes of your test requests.

> **Screenshot reference:** Log Analytics query results showing token usage by team with timestamp buckets.

Step 9 — Go Live (Remove Mock)

When your Foundry Claude deployment is ready:

1. Update the API backend URL:
   - APIM → **APIs** → **Claude Foundry Gateway** → **Settings** tab
   - Update **Web service URL** to: `https://<your-actual-foundry-resource>.services.ai.azure.com/anthropic`
2. Grant the APIM Managed Identity access to Foundry:

```
# Get APIM managed identity object ID
APIM_MI_OID=$(az apim show \
  --name apim-claude-gateway \
  --resource-group rg-ai-gateway \
  --query "identity.principalId" -o tsv)

# Get your Foundry resource ID
FOUNDRY_RESOURCE_ID="/subscriptions/<sub-id>/resourceGroups/<rg>/providers/Mi

# Assign Cognitive Services User role
az role assignment create \
  --assignee $APIM_MI_OID \
  --role "Cognitive Services User" \
  --scope $FOUNDRY_RESOURCE_ID
```

3. Remove the mock operation policy:
   - APIM → **APIs** → **Claude Foundry Gateway** → **POST Messages** → **</>** editor
   - Replace the entire operation policy with:

```
<policies>
  <inbound>
    <base />
  </inbound>
  <outbound>
    <base />
```

```
  </outbound>
  <on-error>
    <base />
  </on-error>
</policies>
```

4. Click **Save** — all traffic now flows through to live Foundry with no other changes required.

Step 10 — Configure Claude Code to Point at APIM

Once verified, redirect Claude Code from Foundry directly to APIM:

Option A — Environment variables:

```
export ANTHROPIC_BASE_URL="https://apim-claude-gateway.azure-api.net/claude/v
export ANTHROPIC_API_KEY="<your-apim-subscription-key>"
```

Option B — Claude Code settings file:

```
// ~/.claude/settings.json
{
  "env": {
    "ANTHROPIC_BASE_URL": "https://apim-claude-gateway.azure-api.net/claude/v
    "ANTHROPIC_API_KEY": "<your-apim-subscription-key>"
  }
}
```

The `ANTHROPIC_API_KEY` value is your **APIM subscription key** (from Step 3.4), not your Foundry API key. No other changes to Claude Code are needed.

Reference: Complete Header Map

Header	Direction	Source	Purpose
`anthropic-version: 2023-06-01`	Inbound → Foundry	API policy	Required by Anthropic API
`x-team-id: <name>`	Caller → APIM	Caller	Metric dimension for chargeback
`anthropic-ratelimit-tokens-limit`	APIM → Caller	Synthesized	Mirrors Anthropic native header
`anthropic-ratelimit-tokens-remaining`	APIM → Caller	Synthesized	Mirrors Anthropic native header
`anthropic-ratelimit-tokens-reset`	APIM → Caller	Synthesized	Mirrors Anthropic native header
`anthropic-ratelimit-requests-limit`	APIM → Caller	Synthesized	Mirrors Anthropic native header
`anthropic-ratelimit-requests-remaining`	APIM → Caller	Synthesized	Mirrors Anthropic native header
`anthropic-ratelimit-requests-reset`	APIM → Caller	Synthesized	Mirrors Anthropic native header
`x-token-usage`	APIM → Caller	Response body	Actual `Nin/Nout` for this request
`x-remaining-tokens`	APIM → Caller	APIM counter	TPM remaining in current window
`x-remaining-monthly`	APIM → Caller	APIM counter	Monthly quota remaining
`x-product-tier`	APIM → Caller	Context	Which tier the subscription is on
`x-gateway`	APIM → Caller	Static	Traceability tag

Header	Direction	Source	Purpose
`x-gateway-request-id`	APIM → Caller	context.RequestId	Cross-service correlation
`Retry-After: 60`	APIM → Caller (429)	API on-error	Standard retry signal

Reference: Values to Customize

Value	Location	Description
`apim-claude-gateway`	All steps	Your APIM instance name
`rg-ai-gateway`	Step 1 / Step 8 CLI	Your resource group name
`claude-sonnet-4-6`	API policy Section F + mock	Your Foundry deployment name
`50000 / 10000 / 2000`	Product policies	TPM limits per tier — adjust to match Foundry quotas
`5000000 / 500000 / 100000`	Product policies	Monthly token quotas per tier
`ClaudeUsage`	API policy Section F	App Insights metric namespace
`<your-foundry-resource>`	Step 2.1 + Step 8	Foundry resource name for backend URL

Downloadable Formats

Keep this Markdown file as the source of truth. For a directly downloadable artifact, also generate an HTML copy after each major edit. From PowerShell 7:

```
$md = 'C:\Users\dasayers\Downloads\apim-claude-foundry-gateway-setup.md'
$html = 'C:\Users\dasayers\Downloads\apim-claude-foundry-gateway-setup.html'
$rendered = ConvertFrom-Markdown -Path $md
```

```
@"
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <title>Azure APIM AI Gateway for Claude on Microsoft Foundry</title>
  <style>
    body { font-family: Segoe UI, Arial, sans-serif; max-width: 980px; margin
    code, pre { font-family: Consolas, monospace; }
    pre { background: #f5f5f5; padding: 16px; overflow-x: auto; border-radius
    table { border-collapse: collapse; width: 100%; }
    th, td { border: 1px solid #d0d0d0; padding: 8px 10px; text-align: left;
    th { background: #f0f0f0; }
  </style>
</head>
<body>
$($rendered.Html)
</body>
</html>
"@ | Set-Content -Path $html -Encoding utf8
```

This produces a standalone HTML file you can email, archive, or open in any browser.

Last updated: April 9, 2026 References: learn.microsoft.com/azure/api-management/genai-gateway-capabilities | platform.claude.com/docs/en/build-with-claude/claude-in-microsoft-foundry